从 TDD 到 EDD

评测驱动的 Data Agent 开发范式

演讲嘉宾：荣柯柯（花名 柯俊）

阿里巴巴 · 高德商业智能部 数据技术专家

目录

CONTENT

01 评测背景介绍

02 选型对比过程

03 方案落地细节

04 方案收益展示

05 优劣、方法论与未来

06 Q & A

01 背景介绍

自我介绍

荣柯柯（花名 柯俊）

阿里巴巴 · 高德商业智能部 数据技术专家

— **深耕数仓与 DataAgent 评测** 主导智能问数评测平台建设，构建四维评测体系与评测 CI

— **NL2SQL 评测方法论** 评测集构建 / 判定规则设计 / 错因归因体系

— **大数据栈**
MaxCompute · AnalyticDB · Flink · StarRocks

— **全栈交付** Python 后端 + React 前端，评测平台从 0 到 1

近两年我的工作可以概括成一句话：**给智能问数 Agent 当测试。**

这些结论来自多大的实践体量

4 个业务方向

统一评测口径

C1 大增长 / C2 打车 / C3 信息业务 / B 端商家，共用一套评分与归因口径

1000+ 题

题库全生命周期

候选 → 正式 → 归档，三阶段状态机 + 先审后归档 + 审计 + 回滚机制。

14 天

评测集滚动更新

四类题源的抽样比例：真实问题 70%、看板与指标库 20%、专家 10%，**两周一轮新旧题轮替 + 指标巡检**，对抗口径漂移

82.6 分

四维能力锚点

D1 任务完成度 45% / D2 推理规划（工具调用）25% / D3 工程效率 15% / D4 安全鲁棒 15%

评测痛点

01 评测集建设不完善

考卷本身不科学，分不出真强还是送分

人工抽检不可持续

发版靠人肉抽 30~50 题（单个业务方向），一轮 1~2 人天
看完就过期，下次发版从头再来

实验室题目太单一

自己出的题永远趋近于满分，线上口语化、
缺主语、时间模糊的问法难以覆盖

口径变更不可感知

上游改一个指标口径，Agent 悄悄答错
没基线、没回归，错几周都没人知道

02 四维评测能力缺失

只看「答对没有」，看不见过程

只有一个笼统分 D1

“完全对 / 部分对 / 跑偏 / 偶然对” 混在一起
好在哪、差在哪，说不清

过程是黑盒 D2

“选表路由、步骤顺序、工具调用与参数”
踩没踩对，全看不见

成本不可知 D3

“同样答对，token / 耗时 成本差 5 倍”
烧不烧得起，答不上来

安全红线被平均 D4

“越权、幻觉、不兜底”被稀释进总分
没有红线一票否决

03 错误分析归因平台化能力缺失

只知考得差，不知怎么变好

只知考得差，找不到病根

错题不归因、不分类
拿不出「该修什么」的清单

同样的错反复犯

失败没沉淀成考题
改一个 bug 冒出两个新 bug

涨跌分不清原因

是 Agent 变了、裁判偏了，
还是口径动了？没有量具校准

失败无法复现

下钻不到某一步操作 / 某条日志（session/trace）
没有最小可归因的闭环

记忆点 代码有 TDD，Data Agent 不能只有「上线见」。

02 选型对比过程

业界应用场景调研对比

方案	题目来源	判定方式	语义上下文	口径判定	能否例行化	借鉴
BIRD 学术基准 · NeurIPS 2023	95 个库、12,751 道题 人工标注，固定题集	EX：执行结果做集合比对 + VES / R-VES 效率分	**有：**逐题外部知识提示	无（**没有独立判定，不是说 BIRD 不涉及口径**）	开发集可自动重跑 测试集需邮件提交	**借：**逐题知识提示 + 效率分
Spider 2.0 学术基准 · ICLR 2025	632 个企业级任务 多步工作流，固定题集	人工编写的任务级评测脚本 按成功率 SR 计分	**有：**元数据、方言文档 与项目代码库	无	可自动重跑 需自备云数仓凭证	**借：**任务级评测脚本
LiveSQLBench 学术基准 · 按版本滚动	按版本滚动发布 上版测试集转为开发集	查询题：Soft EX 评测题：定制测试用例	**有：**分层知识库 HKB （Hierarchical Knowledge Base）	无	可自动重跑 Docker + 评测脚本	**借：**版本滚动+业务规则会变 +定制测试用例
Snowflake Cortex Analyst 工业产品 · Evaluations	人工维护的已验证查询 查询历史推荐需人审	执行 SQL，与标答结果比对 LLM judge 打分，版本锁定	**有：**Semantic View 定义指标、实体与关系	无独立口径分 仅 SQL 正确性分	可定时，可接 CI/CD	**借：**被测题临时移出防泄漏
Databricks Genie 工业产品 · Benchmarks	人工策展，最多 500 题 可从真实会话一键加入	Chat：结果比对 + 人工兜底 Agent：LLM judge	**有：**文本指令、示例 SQL 可信函数、指标视图	**部分：**评测备注引导 LLM judge	后台批跑；Eval API Beta 版，仅评 Chat	**借：**会话入题；等价规则明示
Uber QueryGPT 工业实践 · 2024	QueryGPT 日志真实问题 人工核验意图、表、SQL	5 信号：意图、表重合 能运行、有输出、相似度	**部分：**Workspace 业务域样例 SQL 与表	无	可重复评测看趋势 未公开定时 / CI	**借：**日志取真题；分信号定位
LinkedIn SQL Bot 工业实践 · 2024	领域专家出题 133 题 · 10 个产品域	表/字段召回 · 幻觉率 · 语法 + 专家与 LLM 1-5 分	**有：**知识图谱 字段描述、日志、Wiki	**部分：**专家/LLM 整体打分非专项	持续 benchmark 专家每 3 个月补答案	**借：**一题多解；专家定期校准
我们要的	**真实问题 70%、看板与指标库 20%、专家 10%，评测集滚动更新，对抗口径漂移**	**结果等价 + 口径判定 + LLM 裁判 + 错因归类**	**是**	**是**	**无人值守**	**组合上面各家组件 判定闭环自建（见下页）**

口径判定仅指评测时的独立专项判定；“无 / 部分”不代表产品缺少业务语义能力。

业界应用场景调研对比

判定方法	判的是什么	典型盲区（公开证据）
执行比对 EX、Soft EX BIRD、Spider、**LiveSQLBench** 等	当前评测集上，执行结果是否与标答一致 BIRD 官方实现为集合比对	结果碰巧相同即判对，如都返回 NULL（Spider 官方说明） 集合比对还会忽略行序与重复行
测试套件 Test-Suite EMNLP 2020	为每道题蒸馏出多个变体 预测 SQL 须在所有变体上都与标答一致	有限测试：只是语义准确率的「紧上界」，不是等价证明 仍以标答为准，不懂业务定义
形式化等价 VeriEQL OOPSLA 2024	在完整性约束下，证明或证伪 两条 SQL 的有界等价	只保证「有界」等价，不是全量证明 仍以标答为准，不懂业务定义
LLM 裁判 FLEX NAACL 2025	综合问题、schema、SQL 与执行结果 模拟专家判断对错	与专家一致性 K 从 0.62 升到 0.87，仍非完美 judge 模型与 prompt 须锁版本，否则分数不可比
领域专家人审 LinkedIn 等	意图、口径与多种合法写法是否成立 LinkedIn：约 60% 的题有多个合法答案	成本高、周期长，无法逐日跑 LinkedIn 每 3 个月才复核补充一次

经过借鉴相关行业的经验，我们的组合裁判　四道关依次过滤，每关给出归因标签，专家仲裁只看第 04 关

01　结果等价　判出：结果不一致

容差化结果集比对

行序、重复、数值精度规则事先写明

对应场景：确定性规则

02　口径合同　判出：口径违规

核对公式、粒度、过滤与时间窗

是否符合定义，揪出「碰巧相等」

对应场景：业务语义约束

03　LLM 裁判　判出：多解 / 真错

只裁一题多解与前两关的分歧

judge 模型与 prompt 锁版本

对应场景：一题多解、规则分歧

04　专家仲裁　判出：疑难 / 新口径

只看低置信度、裁判分歧与新口径

结论回灌为新标答

对应场景：低置信度、新口径争议

我们的选型逻辑

四维评分体系与权重

任务完成度 45%
结果对不对：数值等价 / 口径一致 / 维度完整

推理规划 25%
路由对不对：一二级路由命中、选表与指标召回

工程效率 15%
跑不跑得动：SQL 可执行、耗时、重试次数、Token 资源

安全鲁棒 15%
越不越界：SELECT-only、权限隔离、拒答不编造

权重是价值观：宁愿它慢，不能它错。

三条选型判据

01 题必须来自线上真实问法
实验室题「查某表某日 DAU」谁都能答对。线上真实问法「近三个月厂商拉活（渠道唤起）UV 的月均趋势」要先过三关：「近三个月」是哪三个月、UV 能不能按月平均、「趋势」要日粒度还是月粒度。

02 判定必须能回答「为什么错」
单一 EX、Soft EX 只给对/错，只知道错了 20 题，改不动 Agent / 知识库；例如，知道 E5 口径错 42 题、E3 选表错 18 题，才知道先修哪份文档、先补哪类错误。（错因定义见 P14）

03 必须能无人值守例行跑
一次性评测 = 一次性结论。口径漂移是持续发生的，只有评测、巡检例行化才能在漂移当天就发现，而不是等业务方来投诉。

选型后的新问题

三类判定差异：部分来自合法多解或题目 / 标答缺陷

场景 A | 粒度不同

granularity mismatch

Agent 出明细 30 行，标准答案出汇总 3 行

① 两边数值 SUM 之后完全相等

② 行集不同 → EX 直接判 diff

③ 本质：题面没要求细粒度输出，两种粒度都对

④ 如果就这么算， Agent 白背一个错

场景 B | 时间窗模糊

fuzzy time window

题面问「近一个月」，两边理解不同

① Agent 取滚动 30 天，标答取自然月

② 交集区间内数值全等 → 仍判 diff

③ 本质：模糊时间词本身没有唯一解

④ 该修的是题干，不是 Agent

场景 C | 标答自己错了

std answer defect

Gold SQL 把时间窗写死成固定区间

① Agent 按题面答对，反而被判错

② 错误被记到 Agent 头上，进了归因报告

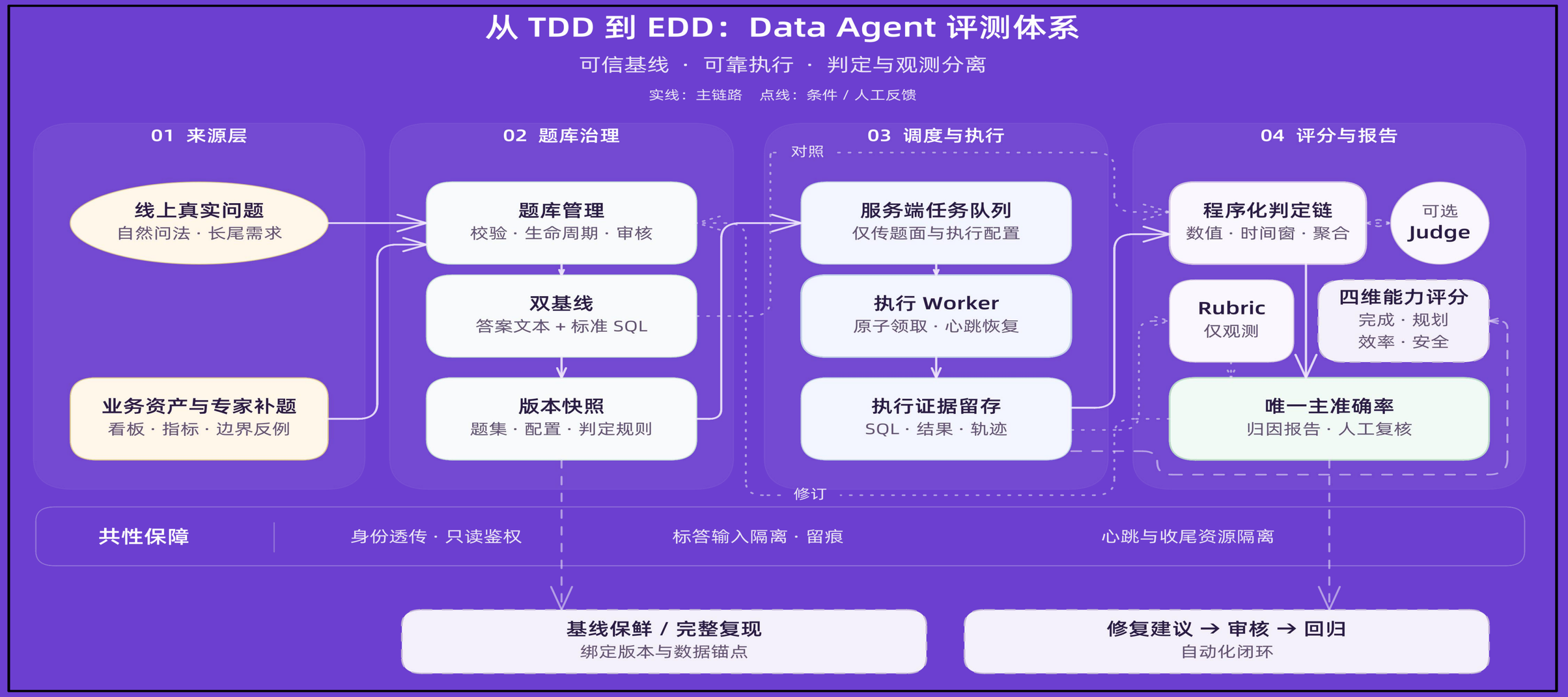
③ 本质：这是评测器的锅，不是 Agent 的锅

结论 EX、Soft EX 是判定的起点，不是终点，我们后来把判定做成了 9 条确定性出口，P13 展开。

记忆点 EDD 的价值不仅是让 Agent 获得更好看的分数，而是让团队能够基于证据决定该改什么，并识别“分数变化”究竟来自能力、样本、规则还是环境。

03 方案落地细节

架构总览：建—管—CI—归因 四阶段流水线



架构总览：建—管—CI—归因 四阶段流水线

一页定全景：每个阶段都对应 EDD 开发环的一个动作

01 建
Build

① 线上真实问题 → 题干标准化
② 每题双基线：标准答案 + 标准 SQL
③ 11 列标注：域 / 负责人 / 难度 / 来源 / 一二级路由
④ 难度分层 + 复杂度加权抽样
⑤ 专家补齐边界题

EDD · 写用例

02 管
Manage

① 14 天滚动更新：新题入、旧题退
② 版本快照：结论绑定 snapshot 可回溯
③ 三阶段状态机：候选 → 正式 → 归档
④ 新旧题分离统计，回归与探边分开看

EDD · 维护用例库

03 CI
Run

① N 路并行 + 错峰调度
② 契约驱动 + 心跳保活 + 自动重跑
③ 答案隔离：主代理读不到标准答案
④ 断点续跑，每题完成即落盘

EDD · 自动跑用例

04 归因
Attribute

① 9 条确定性判定出口
② E1–E9 两级错因 + 帕累托 + 归责
③ 原始 / 剔噪双准确率口径
④ 反哺 Skill 文档与指标知识

EDD · 看红绿灯 + 重构

支撑层：这三件事不做，上面四段都跑不起来

执行网关

ODPS / ADB 统一入口：SELECT-only 白名单 + 用户身份透传 + 事前鉴权。评测跑的是用户自己的权限，越权直接拒。

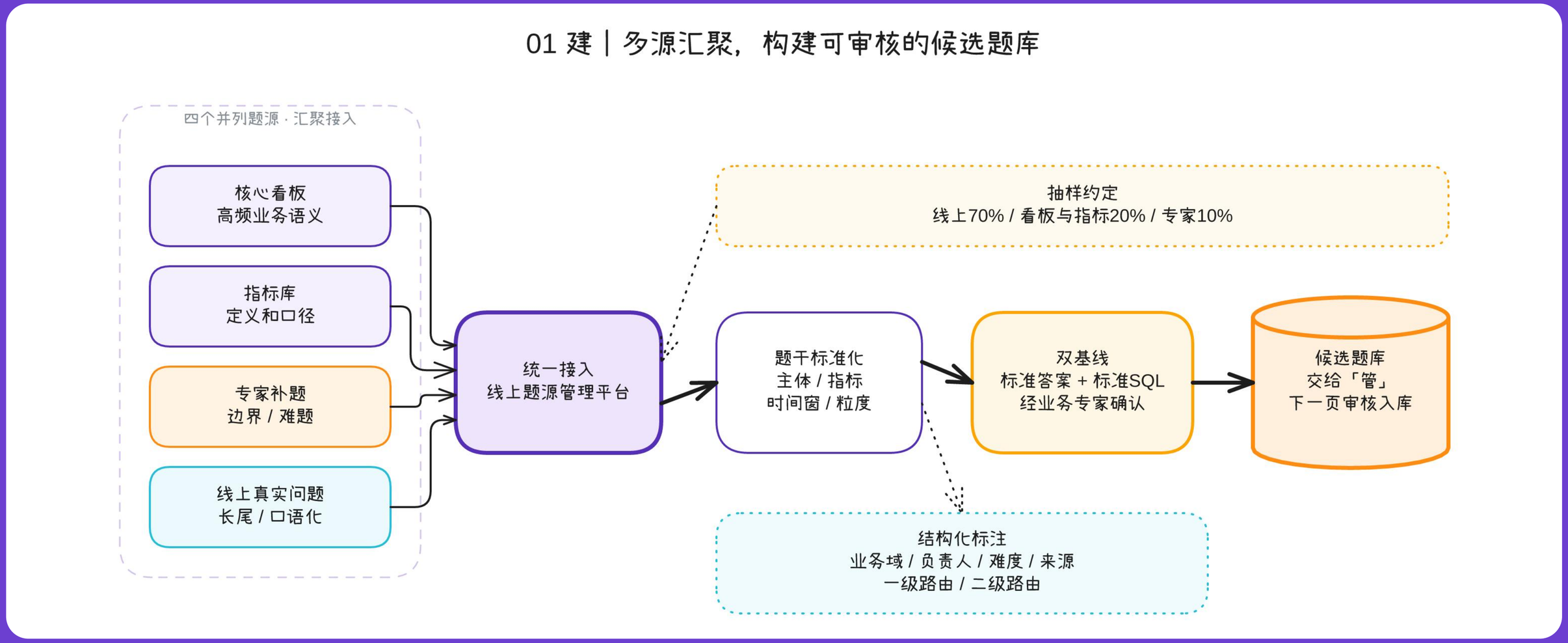
答案隔离

标准答案全局唯一文件，主代理只读脱敏题干；所有对比在子进程内完成，防止标答泄进模型上下文。

完整性校验

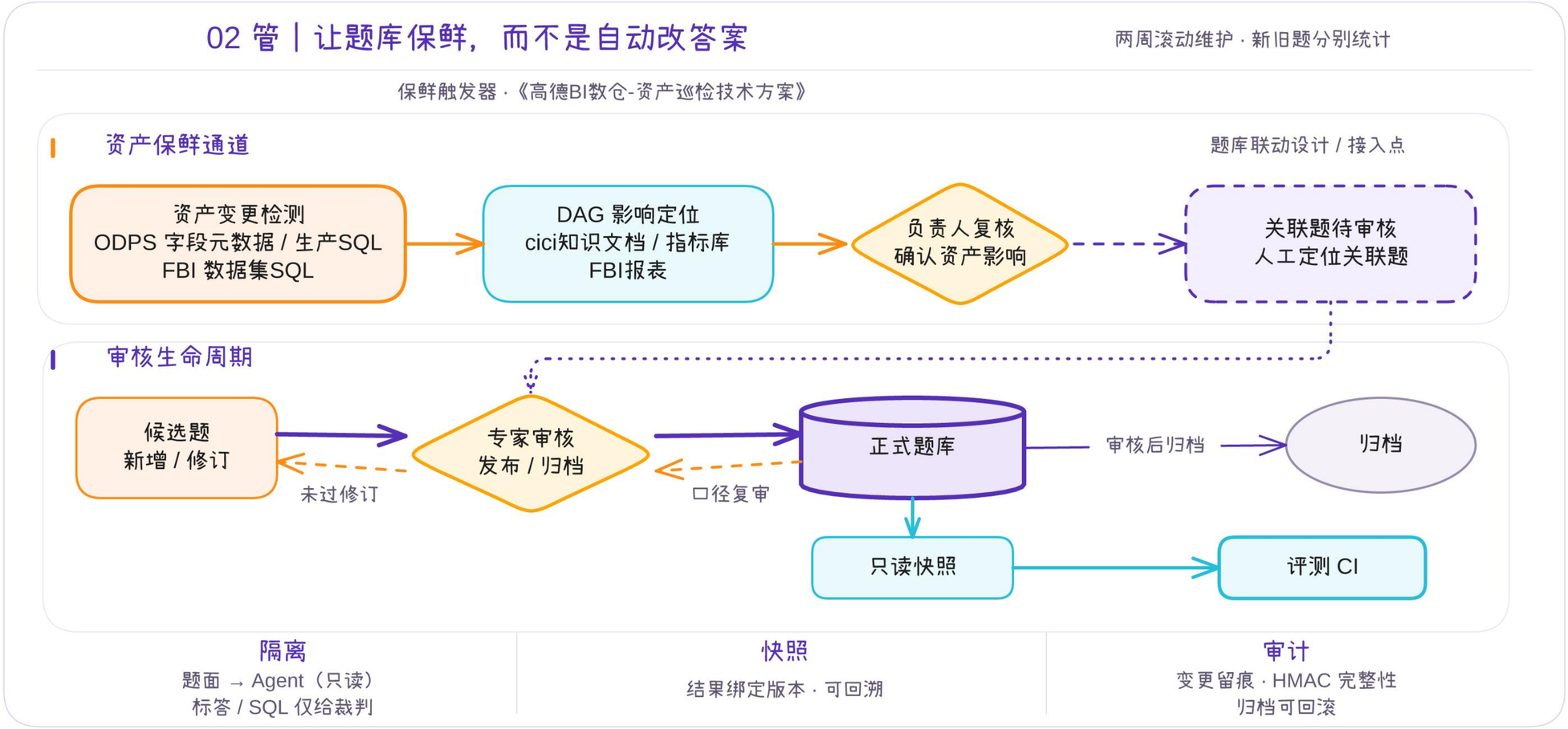
HMAC 签名防篡改 + 三方交叉校验（契约期望数 / 状态上报数 / 结果条目数）。对不上就不出报告——宁可没结论，不要错结论。

01 建：四源汇聚，把真实问题变成评测题



建，不是让 Agent 给自己出题，而是把真实业务问题沉淀成可审核的候选题。

02 管：变更感知，让题库持续可信



管，不只是更新题目；还要锁定口径、隔离答案，让每轮评测可复现、可回溯。

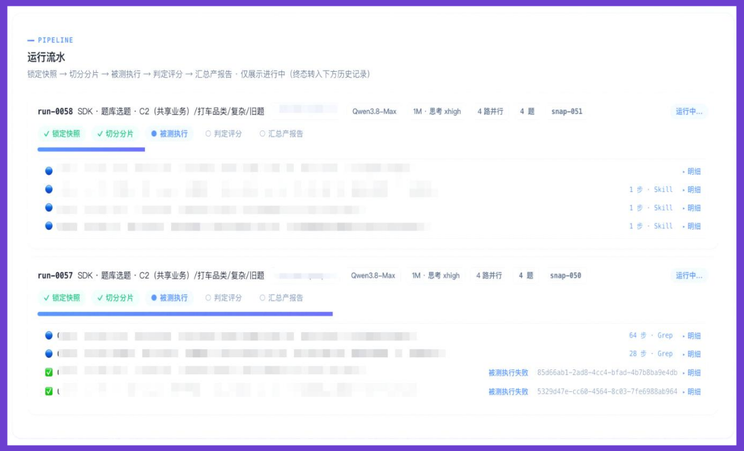
CI：无人值守的并行评测



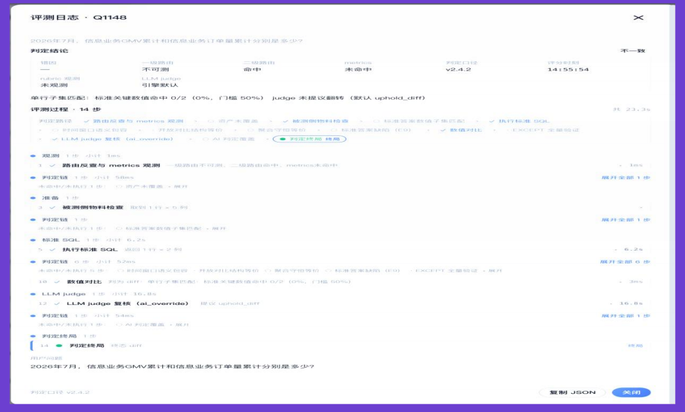
Skill 包管理



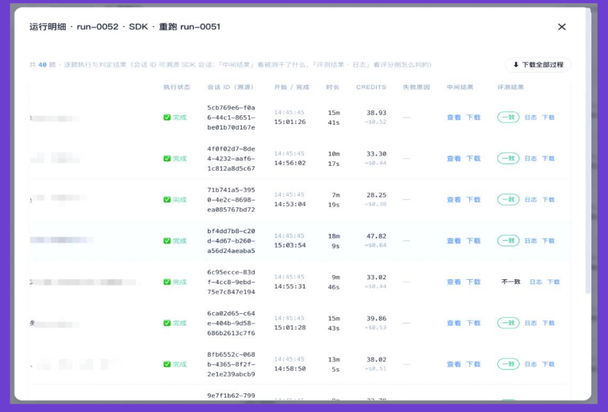
新建评测任务



流水运行



评测日志



运行明细



评测历史记录

CI：无人值守的并行评测

N+1 定时任务：N 个执行 + 1 个汇总轮询

① 切分

评测集 → 均分 N 片（余数后置）；11 列强校验不通过立即终止；**标准答案抽离成全局唯一文件**，分片里只留脱敏题干

② 派发

- **每片预写 _eval_contract.json** 路径单一来源：worker 只从契约读，禁止从消息里二次解析
- **N 个任务错峰 2 分钟依次启动** 避免同时抢 ODPS 资源；每个任务独立执行自己那一片

③ 收口

- **第 N+1 个任务每 2 分钟轮询一次** 状态机 running → done / failed / stale，全部终态才出报告
- **完整性交叉校验** 契约期望数 vs 状态上报数 vs 结果实际条目数，三者必须一致
- **不完整分片自动重跑（≤3 次）** 达上限的以 incomplete 纳入报告，不阻塞汇总
- **汇总 + 归因看板后自我 disable** 不留僵尸定时任务

让它敢无人值守的四条硬机制

心跳必写

每完成一题就更新状态文件；心跳延迟超过 600 秒判 stale，触发重派

答案隔离

全局唯一 answers_locked 文件，禁止在任何分片目录复制副本

真实调用红线

必须真实调用被测 Skill。禁止复制标准 SQL 走捷径，违反则整批结果作废——Agent 是会偷懒的

并发可调

4–16 路按 ODPS 配额调整；断点续跑，每题完成即落盘

效果 **500 题评测 4 小时+ → 约 30 分钟**（单台机器16 路并行） · **评测周期 24h → 12h** · **人力成本降约 30%**

归因：把判定从提示词搬进确定性软件

9 条判定出口，全部脚本化、可回归、带审计字段

#	判定出口	触发条件	判定结果 / 审计字段
1	资产未覆盖	标准答案整格标「未覆盖 / 暂无 / TBD」（保守词典 · 整格精确匹配）	asset_uncovered → 计入分母算失败，E8
2	标答优先对比	结果与标准答案关键数值子集命中率 ≥ 50% 且至少 1 个	match 终局；对比在子进程内完成（不满足100%的，走LLM 裁判）
3	时间窗语义包容	含模糊时间词 + 两窗连续包含 + 边界差 ≤ 容差 + 交集数值全等	match；time_window_tolerance
4	开放对比题结构等价	「X 和 Y 有什么区别」句式 + 双主体覆盖 + 各 ≥2 分组 + 方向一致	match；open_question_structural
5	聚合守恒等价	细粒度经 SUM/AVG/MAX/MIN/COUNT 可推导粗粒度 ≥80% 核心值	match；aggregation_conservation
6	标答缺陷剔除	四条护栏 G1–G4 全过（题要明细 / std 仅汇总 / skill 有明细 / 重叠 ≥90%）	E9 → 从真实准确率分母剔除
7	口径歧义待复核	疑似标答口径缺陷但护栏未全通过	仍计错 + 报告显著提示，人工定案
8	EXCEPT 双向差集	std_minus_skill 与 skill_minus_std 均为 0	both_except_zero 才算等价
9	路由准确率	一级路由域 / 二级路由表分别比对标注值	route_stats + 路由未命中明细

三条不可退让的红线

红线 1

必须真实调用 Skill

禁止复制标准 SQL、批量推断、人工代写。违反则整批结果无效。

红线 2

资产未覆盖绝不判 match

无资产的问题照样计入分母算失败，同时观测 Skill 是拒答还是编造。

红线 3

标答绝不进主代理上下文

主代理只读脱敏题干，对比都在子进程里做，防止照着答案写 SQL。

每条豁免都配审计阈值，防止规则被滥用： 资产未覆盖占比 > 15% 报警 · 编造行为占比 > 30% 单独报警 · 聚合守恒命中 > 30% 说明标准 SQL 粒度约定与真实问法脱节 · 疑似标答缺陷 > 20% 报警

归因：E1–E9 不是我们拍脑袋分的

每一类错因都能在国内外工业实践 / 权威基准里找到对应的一手证据

错因	我们的定义	业界对应证据（一手信源）	修复动作落在哪
E1	意图理解 / 歧义	Genie：业务黑话误解、忽略指令；Uber：prompt enhancer 处理欠明确输入	题干标准化 + 澄清话术
E2	路由分发错（一级路由）	Genie：多空间路由与指令遵循问题	路由规则与域索引文档
E3	选表错（二级路由 / 表链接）	Spider 2.0：schema linking 占 27.6%（表 10.1 / 列 16.6）	补表级文档与示例 SQL
E4	字段 / 取值 grounding 错	Genie 把「值不匹配过滤」独立成类：按 California 过滤但库里存 CA	值字典 + 枚举样例
E5	指标口径 / 业务语义错	Genie：指标计算错 + 时间计算错；BIRD：外部业务知识缺口	结构化指标知识（本场重点）
E6	SQL 逻辑错	Spider 2.0：查询规划 17.7%、JOIN 8.3%	模板化 SQL + 示例增强
E7	语法 / 方言错	Spider 2.0：方言函数错 10.3%	方言约束与预编译校验
E8	幻觉 / 资产缺失	BIRD 噪声研究：问题无法映射到 DB	资产未覆盖检测 + 诚实拒答
E9	结果解读 / 评测噪声	BIRD-Bench 噪声研究六类噪声之一就是「错误的 gold SQL」	标答缺陷护栏 + 剔噪口径

必须提醒 Spider 2.0 / BIRD 的百分比是学术基准上的分布，不能当作线上分布的预期值；国内厂商暂无通过验证的公开归因分类，这套框架的国外根基强于国内。

记忆点 **我们不是发明了一套分类，是把工业界已验证的归因维度落到自己的数据资产上。**

04 方案收益展示

量化效果：只讲 4 个数

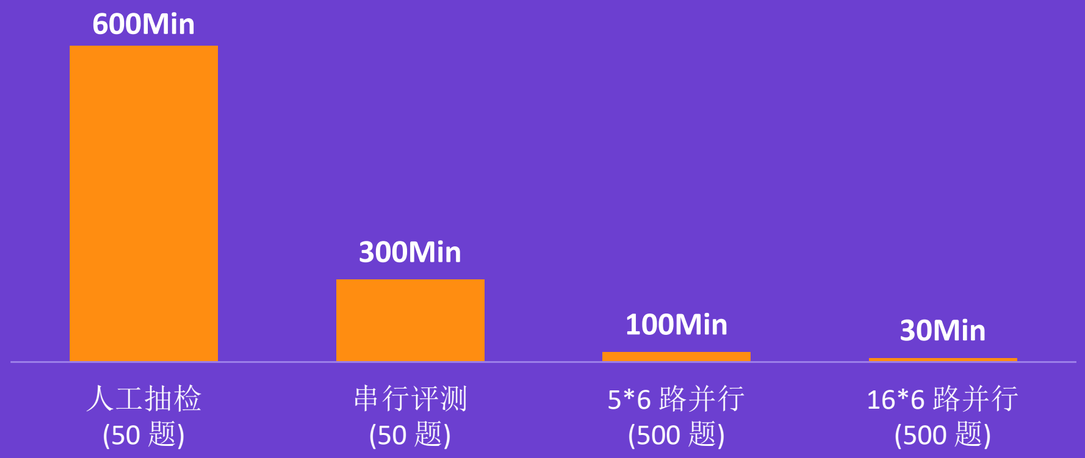
95%
四个业务方向合并准确率

30 min
500 题评测总耗时
原串行需 4 小时以上；并行全开时压缩至约 30 分钟，最高支持 16 路并发（单台服务器）

−30%
评测人力成本
评测周期 24h → 12h；无需人盯，失败分片自动重跑，无新增人力投入

1000+ 题
累计维护 · 全生命周期
4 个业务方向统一口径，14 天滚动更新，题库管理已生产可用

评测耗时：从人肉到无人值守



准确率有两个口径，报告同时给出

原始准确率
全样本口径：match 题数 ÷ 全部题数。含标准答案本身有缺陷的题，会系统性低估 Skill 的真实水平。对外汇报用这个数最保守。

真实准确率（剔噪）
命中 E9 标答缺陷护栏的题从分母剔除（分母 −1），衡量的是 Skill 自身能力。两个数一起看，才能分清「Agent 错」和「评测错」。

一道真实的题：两个口径，差 0.5%

用户问题 某日 C3 域某经营类目的「动销 POI 数」是多少？ 来源：线上真实评测 · 难度 中 · 一级路由 trade

六步链路定位：口径分歧始于选表与指标选择，执行成功不等于语义正确

1. ✓ **意图理解** — 问的是动销 POI 数量
2. ✓ **路由分发** — 一级路由命中 trade
3. ? **选表与指标** — 供给 / 交易口径分歧
4. ? **SQL 生成** — 语法可执行，语义待定
5. ✓ **执行** — ODPS 成功返回单值
6. ✓ **结果解读** — 忠实输出所选口径

Skill 的答案 | 供给域「真动销」口径

```
SELECT transaction_move_sell_poi_cnt_1d
FROM   ads_..._merchant_supply_index_di    -- 供给域
WHERE  ds = '2026xxxx'
  AND  business_dept_l2 = '<某类目>'
```

62,318

标准答案 | 交易域「下单 POI」口径

```
SELECT kpi_order_poi_cnt_1d
FROM   ads_..._order_biz_trade_overview_di  -- 交易域
WHERE  ds = '2026xxxx'
  AND  business_dept_l2 = '<某类目>'
```

62,650

结果差异与责任分开判

- 数值相差 332，相对偏差 0.53%
- 聚合守恒 / 时间窗包容 / 开放题结构等价 三项豁免均未命中
- 标答缺陷护栏 G1–G4 未全过 → 不做剔噪
- 判定 diff | 根因 E5（口径分歧）| 归责待复核

评测揭示口径治理问题

「动销」在供给域与交易域有不同定义，分歧已进入选表与指标选择。执行和解读可以正确，但业务语义仍待确认；与标答不一致，不直接等同于 Agent 错误。

反哺动作 业务负责人确认适用域、公式、时间窗与粒度 → **更新**指标知识和合法数据源映射 → 回归验证。

踩过的坑：评测器自己错了，把对的判成了错的

一个真实案例走完四步：C1 域 · 渠道拉活 · 「厂商拉活 UV 月均趋势」

01 现象

该题判 diff

- 标答关键数值子集只命中 8/20（40%），低于 50% 阈值
- 报告里它被自动归到 E1 意图理解错
- 从数据上看，这就是一道普通错题

02 第一反应

以为又是 Skill 答错了

- 差点写进归因报告，反馈给 Skill 负责人去改意图理解
- 如果就这么过了，我们会去修一个不存在的问题
- 更糟的是，准确率被无理由拉低

03 复核

两边其实是同表同口径

- 同一张底表、同样的过滤条件（渠道 / 生态类型一致）
- 唯一差别是时间窗：gold SQL 把窗口写死了
- 题面问「月均趋势」，复核确认应逐月输出

04 定案

标答缺陷，不是 Skill 错

- 人工复核后 diff → match 改判，记标答缺陷
- 真正该修的是评测集，不是 Agent
- 不剔除的话，会污染整条趋势线

这个坑让我们补上了什么

— **新增 E9 标答缺陷类 + 四条护栏 G1–G4** 自动检出、自动从真实准确率分母剔除，不再依赖人工逐题标注

— **新增口径歧义待复核标记** 护栏没全过的可疑题仍计错，但报告显著提示「疑似标答口径缺陷」

— **准确率改成双口径输出** 原始准确率 与 剔噪真实准确率同时呈现，不再只报一个数

尚待解决的两个问题

— **「Skill 真实错误率」与「评测不可判率」还分不干净** 标答会随口径演进悄悄失效；单实体标答覆盖不了多实体问法；护栏也有假阴性

— **基线健康度如何低成本持续保鲜，仍无好解法** 目前只能靠人工抽检兜底误判归因，而这本身又变成了新的人力消耗

交流邀请：若有低成本标答保鲜方案，欢迎会后交流。

记忆点 **谁来评测「评测」——这是我们目前最没解的问题。**

05 优劣、方法论与未来

客观说：这套东西的优势与代价

想抄这套方案的团队，请先看右边一列

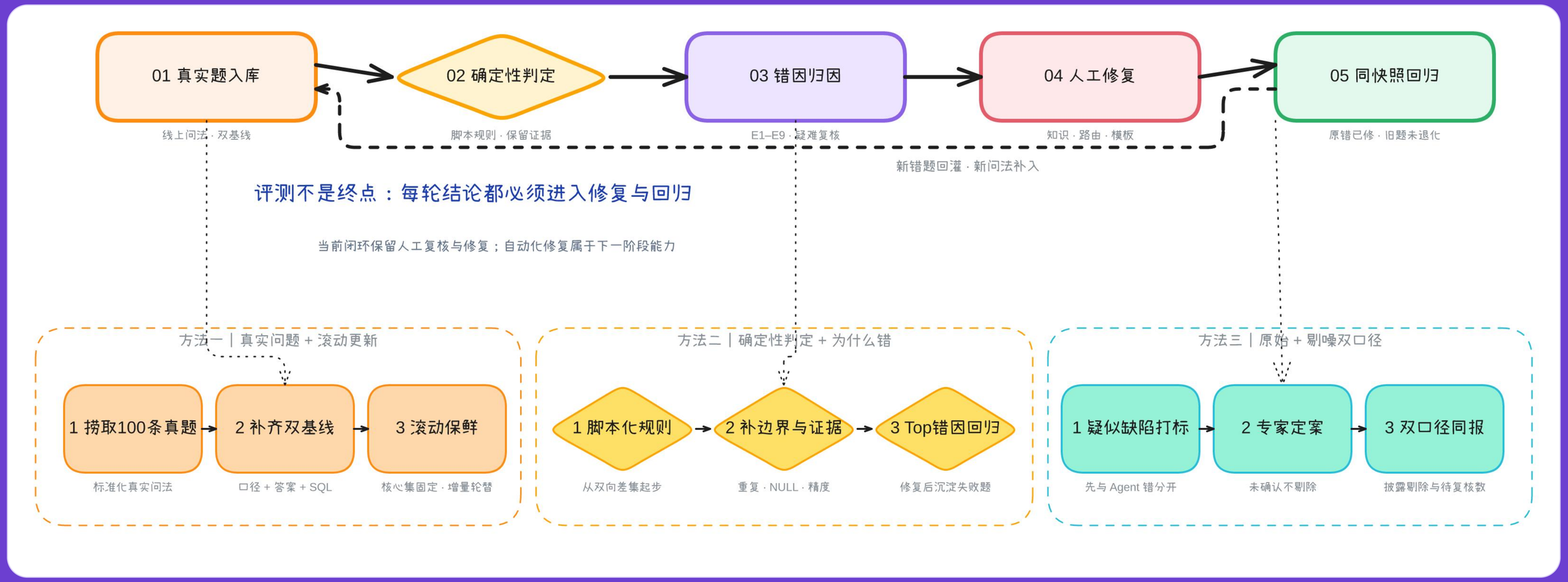
维度	优势 · 我们确实拿到了什么	代价 / 前提 · 你必须先付出什么
结论质量	结论可回归、可解释：E1–E9 两级错因 + 原始/剔噪双准确率口径，涨跌能逐题溯源	判定规则本身要养：9 条判定与观测规则部分存在假阳性风险，必须配审计阈值做反向监控
人力投入	无人值守例行化，评测周期 24h → 12h，人力成本降约 30%，无新增人力	工程复杂度高：契约驱动 / 状态机 / 心跳 / 自动重跑 / 答案隔离，缺一个就不敢无人值守
评测集	题源 100% 来自线上真实问题 + 14 天滚动更新，抗口径漂移	标答成本高：标准答案文本与标准 SQL 双基线要人写、要持续保鲜，这是最重的一块
改进闭环	归因能定位到「为什么错」，能把 Skill/知识库 该改哪份文档、该补哪个值字典指出来	人工闭环已形成，自动修复仍未打通：归因 → 修复仍需人工复核与处理。
安全合规	SELECT-only 白名单 + 用户身份透传 + 答案隔离 + HMAC 完整性校验	权限与执行环境改造是硬前置，不是可选项；上线前必须先把权限方案谈通

一句话建议 **如果你的团队只能投入一个人，先做「建」和「归因」，别先做 CI。**

有真实题、有错因分类，哪怕手动跑，价值也已经出来了；CI 是把已验证的价值放大，它本身不产生价值。

记忆点 **能例行化的评测才有复利，但复利要先付工程本金。**

三条可以直接带走的方法论



记忆点　真实问题入题 → 确定性判定定位 → 修复后回归验证，**EDD** 才形成开发闭环。

从 TDD 到 EDD：把「测试」补回开发流程

回扣主旨

Data **Agent 同样需要可回归的测试；**现在这四个动作已补齐：

写用例
建 · 线上真实问题 + 双基线

▼

维护用例库
管 · 14 天滚动 + 版本快照

▼

自动跑用例
CI · 并行调度 + 无人值守

▼

看红绿灯并重构
归因 · E1–E9 + 反哺 Skill

三个结论

01　评测是 Agent 迭代的基础设施

不是发版前的一道审批。没有它，每次迭代都是赌博。

02　真实问题 + 滚动更新

这是对抗口径漂移目前唯一有效的手段。

03　判定要细到「为什么错」

评测才能从「打分」升级为「改进」。

未来三件事

看板化
结论进大盘：趋势可比、版本可对照、退化能自动报警

知识化
口径知识结构化沉淀，自动反哺 Skill 的 SQL 生成——这是「评测 → 知识 → 准确率」双向反馈的关键一环

硬隔离
标准答案与执行环境物理隔离，把「不能作弊」从流程约束变成架构约束

留两个还没解的问题　① 谁来评测「评测」；　② 归因 → 修复的闭环如何自动化。　欢迎有同类实践的团队会后交流。

记忆点　**TDD 让我们敢重构代码，EDD 要让我们敢重构 Agent。**

THANKS

Q & A

演讲嘉宾：荣柯柯（花名 柯俊）

阿里巴巴 · 高德商业智能部 数据技术专家